Mock Midterm: The Riddle Board

A practice coding exam: a simple, static React app with CDN

This is a **practice exam**. It has the same shape, the same rules and the same marking scheme as the programming part of the real midterm, so you can find out where you stand before the day. Give yourself **90 minutes**, work alone, and mark yourself with the checklist in Section 8.

You will build the **Riddle Board**: one page that shows nine riddles, each on its own card, with a header at the top and today's date in a footer. There are no buttons, no state and no hooks. It is a **static** React app: everything you need was in Module 4 (*Movie Night*) and in Activities 1 to 4 of the React Code Lab.

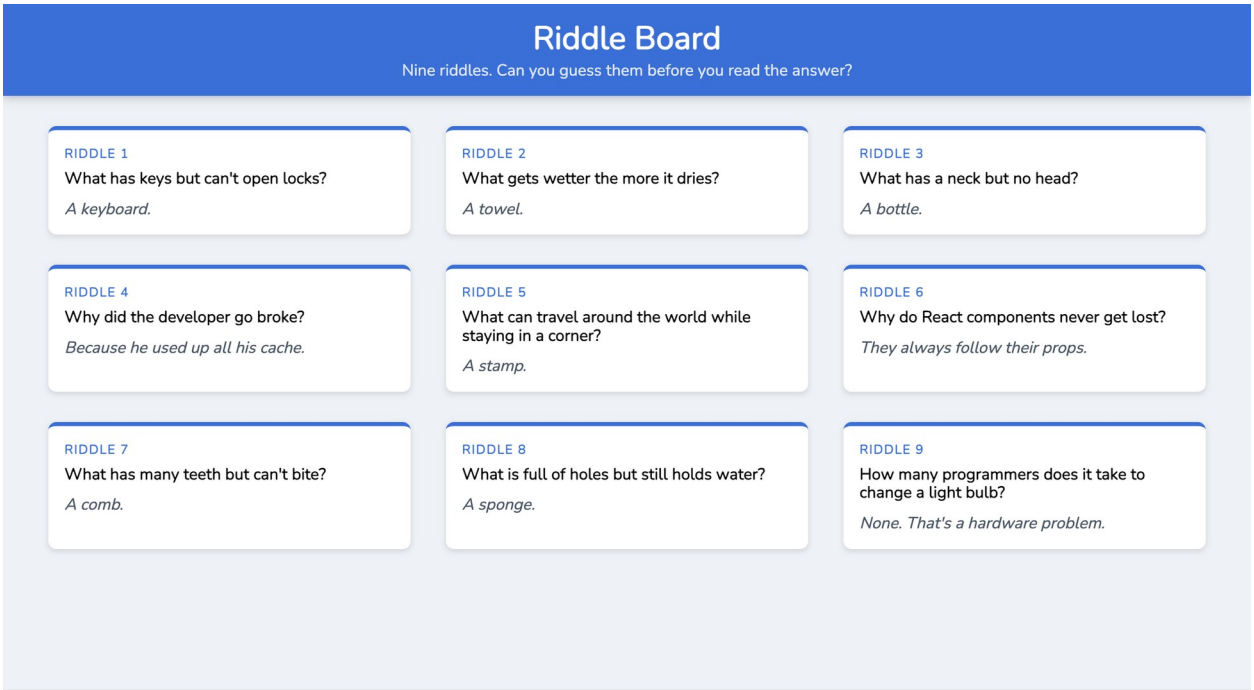


Figure 1 — the goal: the finished Riddle Board

> **HOW TO USE THIS DOCUMENT**
> Read Sections 1 to 4 first: they tell you *what* to build. Section 5 gives a step-by-step plan with hints. The hints show the *idea* with a different example (fruits, not riddles), so you still write the code yourself. Section 6 lists the most common errors and how to fix them. Try each step on your own before you read its hint.

1. The exam rules (same as the real midterm)

The real midterm has two parts:

Part	Where	Marks
Knowledge test (multiple choice, short answer)	Blackboard	5
Programming challenge (an app like this one)	Submitted as a .zip in Blackboard	25

For the programming challenge:

- Build your app in **one folder**. Zip the folder and submit the .zip through the exam's assignment link in Blackboard.
- Your instructor will unzip your folder and open it. **If the page does not show up, the mark is zero.** Test your app before you submit, and test the zip too (Section 7).
- Submitting the empty starter files, or something unrelated, to try for a partial mark costs you **3 marks** (−3).

---

2. What you are given

Download and unzip `riddle-board-start.zip`. It contains:

File	What it is	Do you change it?
`index.html`	The page. It has the `<div id="root">` and the CSS links.	**Yes**: add the `<script>` tags
`riddles.js`	The data: nine riddles in a variable called `riddles`	No
`styles.css`, **`normalize.css`**	The finished styles. Every class you need is already there.	No
`mockup.html`	The design in plain HTML, so you can see which classes go where	No (just open it)
`components/Header.jsx`	The header component. Already finished.	No
`components/Riddle.jsx`	One riddle card, but with placeholder text	**Yes**: use props
`components/Footer.jsx`	The footer, but with placeholder text for the date	**Yes**: show the date
`app.js`	Empty except for TODO comments	**Yes**: write `App` and render it

This is what the data in `riddles.js` looks like (two of the nine riddles shown):

```
const riddles = {
  "riddles": [
    {
      "id": 1,
      "question": "What has keys but can't open locks?",
      "answer": "A keyboard."
    },
    {
      "id": 2,
      "question": "What gets wetter the more it dries?",
      "answer": "A towel."
    },
    ...
  ]
};
```

> **READ THIS CAREFULLY**
> The variable is called `riddles`, and the array is inside it, also under the name `riddles`. So the array of nine riddles is `riddles.riddles`. Type `riddles.riddles` in the browser console once your page loads `riddles.js`: you should see an array with 9 items.

3. What to build

The page has **four components**. Figure 2 shows which part of the page each one draws.

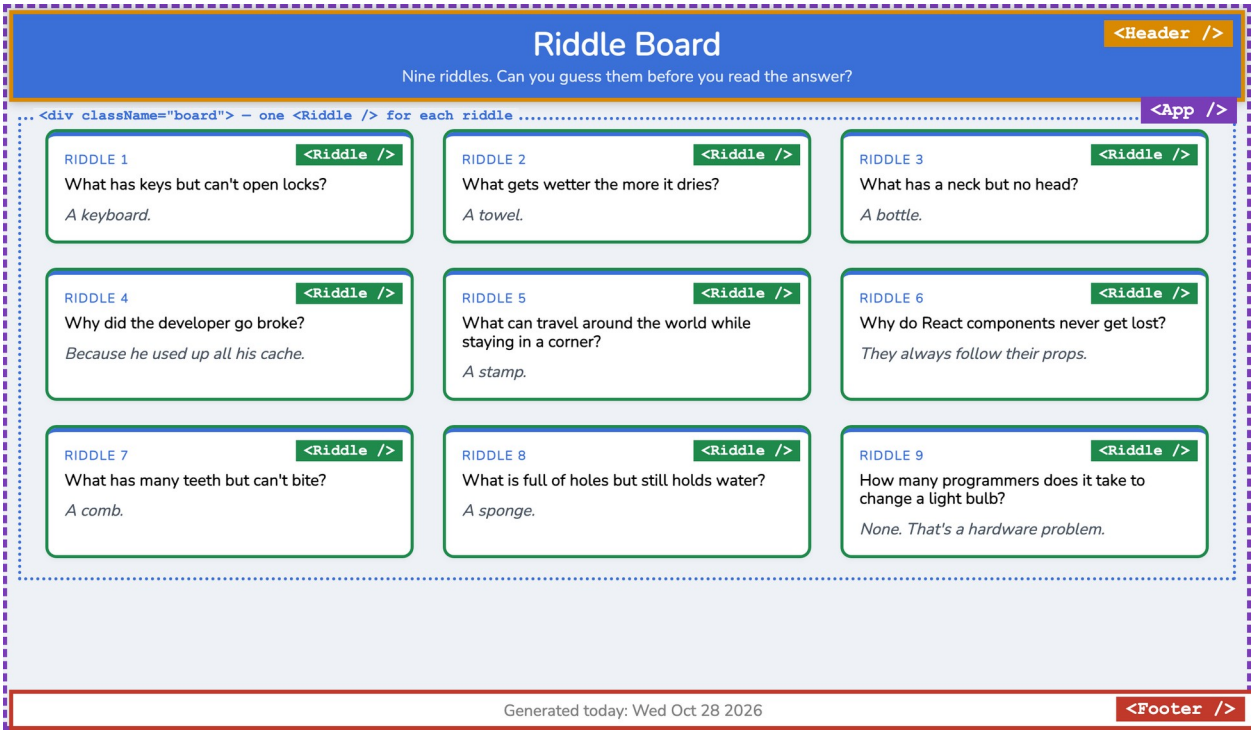


Figure 2 — the four components of the Riddle Board

Part	File	What it shows
`App`	`app.js`	Everything: the `Header`, then a `<div className="board">` with **one `Riddle` for each of the 9 riddles**, then the `Footer`
`Header`	`components/Header.jsx`	The blue bar with the title (already done)
`Riddle`	`components/Riddle.jsx`	**One** card: its number, its question and its answer
`Footer`	`components/Footer.jsx`	"Generated today:" followed by **today's date**

Each card uses these classes from `styles.css` (look at `mockup.html` to see them in place):

```
<div className="card">
  <span className="card-number">Riddle 1</span>
  <p className="question">What has keys but can't open locks?</p>
  <p className="answer">A keyboard.</p>
</div>
```

---

4. Requirements

1. Build the app with **React from a CDN** (no Vite, no `npm install`). Use the same three scripts you used in Module 4: React, ReactDOM and Babel.
2. Split the app into **four components**: `App`, `Header`, `Riddle` and `Footer`.
3. **Read the riddles from `riddles.js`**: load it with a `<script>` tag and use the `riddles` variable. Do not copy the data into your own code.
4. Show **all nine** riddles, using a **loop** (`map`), not nine hand-written cards.
5. Show **today's date** in the footer. It must be the real date, not typed in.
6. **No errors** in the browser console, and **no React warnings** (the one Babel warning, "You are using the in-browser Babel transformer", is expected and is fine).

---

5. A step-by-step plan, with hints

Work in this order. After each step, save, refresh the browser and check that the step works before you move on. If you skip the checks, one small typo early on can cost you the whole exam.

Step 1 — Open the project with Live Server (5 min)

1. Unzip the starter, open the folder in VS Code, and open `index.html` with **Live Server** (right-click → *Open with Live Server*).
2. Open the browser's **Developer Tools** (F12, or Cmd+Option+I on a Mac) and keep the **Console** tab open for the whole exam. Every error shows up there.
3. Also open `mockup.html` in the browser, to see what you are aiming for.

> **WHY LIVE SERVER?**
> Babel loads your `.jsx` and `app.js` files over the network. If you double-click `index.html`, the address starts with `file://` and the browser blocks those loads: you get a blank page. The address must start with `http://`.

Step 2 — Add the scripts to index.html (10 min)

Replace the TODO comments in `index.html` with `<script>` tags, in this order:

1. React (CDN)
2. ReactDOM (CDN)
3. Babel (CDN)
4. `riddles.js` — a **normal** script (no `type`), because it is plain JavaScript
5. `components/Header.jsx`, `components/Riddle.jsx`, `components/Footer.jsx` — Babel scripts
6. `app.js` — a Babel script, and the **last** one, because it uses all the others

Hint. The three CDN lines are in the `index.html` of *Movie Night* (Module 4) and in Code Lab Activity 1. A file Babel must translate needs `type="text/babel"`; a plain JS file does not:

```
<script src="./fruits.js"></script>
<script type="text/babel" src="./components/Fruit.jsx"></script>
```

Check. Refresh. The page is still empty, but the Console shows **no red errors**, and typing `riddles.riddles.length` in the Console prints `9`.

Step 3 — Render App with the header and footer (10 min)

In `app.js`, write a function component called `App` that returns the `Header`, an empty `<div className="board"></div>`, and the `Footer`. Then render `App` into the `root` div.

Hint. A component must return **one** outer element, so wrap the three parts in a `<div>`. Rendering with React 18 takes two lines; you wrote them in every Module 4 exercise:

```
const root = ReactDOM.createRoot(document.getElementById("..."));
root.render(<SomeComponent />);
```

Check. You see the blue header and, at the bottom, "Generated today: current date".

Step 4 — Show one riddle card with props (15 min)

Make `Riddle` show the riddle it is **given**, instead of the placeholder text.

1. In `Riddle.jsx`, let the function take `props`.
2. Replace `Riddle 1`, `Riddle question` and `Riddle answer` with `{ }` expressions that read from the props.
3. To test it, put **one** card in the board in `App`, and pass it the first riddle, `riddles.riddles[0]`.

Hint. The same idea with fruits: the parent passes a whole object as one prop, and the child reads its fields.

```
function Fruit(props) {
  return <p className="name">{props.fruit.name}: {props.fruit.price}</p>;
}

// in the parent:
<Fruit fruit={fruits.fruits[0]} />
```

Check. One card shows "RIDDLE 1", the keyboard question and "A keyboard.".

Step 5 — Show all nine with map() and key (15 min)

Replace the single test card with a **loop**: one `Riddle` for every riddle in the array.

Hint. `map()` turns each item of an array into a piece of JSX. Each item needs a `key` that is unique and stable; the data already has one for you (look at the fields in `riddles.js`).

```
<ul>
  {fruits.fruits.map((fruit) => (
    <Fruit key={fruit.id} fruit={fruit} />
  ))}
</ul>
```

This is the same thing you did to show the list of movies in *Movie Night* (Code Lab Activity 4).

Check. Nine cards, numbered 1 to 9, in a grid of three by three on a wide screen, and the Console has **no** "unique key prop" warning.

Step 6 — Today's date in the footer (5 min)

Replace "current date" in `Footer.jsx` with the real date.

Hint. `new Date()` gives you the date and time right now. Store it in a variable before the `return`, then put it in the JSX with `{ }`. A `Date` object cannot go straight into JSX: turn it into text first. Try these in the Console and pick the one you like:

```
new Date().toDateString()          // "Wed Oct 28 2026"
new Date().toLocaleDateString()    // "10/28/2026"
```

Check. The footer reads, for example, "Generated today: Wed Oct 28 2026", and tomorrow it will show tomorrow's date.

Step 7 — Clean up (5 min)

- Delete test code, `console.log` lines and code you commented out.
- Add a short comment above each component saying what it shows.
- Make sure all four files are saved, refresh, and look at the Console one last time.

---

6. Common errors and how to fix them

What you see	The usual cause	The fix
A blank page and no error	You opened the file directly (`file://...`)	Open it with **Live Server** (`http://...`)
`React is not defined`	A Babel script runs before the React CDN script	Put the three CDN scripts **first**
`riddles is not defined`	`riddles.js` is missing, misspelled, or loaded after `app.js`	Load `riddles.js` **before** the Babel scripts; check the file name
`Header is not defined` (or **`Riddle`**, **`Footer`**)	The component file is not loaded, or is loaded after `app.js`	Add its `<script type="text/babel">` **before** `app.js`
`Unexpected token '<'`	A file with JSX is loaded **without** `type="text/babel"`	Add `type="text/babel"` to that script tag
`Cannot read properties of undefined (reading 'map')`	You called `riddles.map(...)`	The array is `riddles.riddles`
`Each child in a list should have a unique "key" prop`	The `key` is missing, or is on the wrong element	Put `key={...}` on the `<Riddle />` that `map()` returns
`Objects are not valid as a React child`	You put a whole object (a riddle, or `new Date()`) inside `{ }`	Show one field (`.question`), or turn the date into text
The cards have no style	You wrote `class=` instead of `className=`	In JSX it is always `className`
`Adjacent JSX elements must be wrapped`	A component returns two elements side by side	Wrap them in one `<div>`
Only one card, or the same card nine times	You passed `riddles.riddles[0]` inside `map()`	Pass the item `map()` gives you

> **READ THE ERROR, THEN THE FILE AND LINE**
> The Console says which file and line went wrong. Click it to jump there. Most errors in this exam are a name typed differently in two places, or scripts in the wrong order.

7. Before you submit

1. Close the browser tab, open `index.html` again with Live Server, and check the page one last time.
2. Zip the **whole folder** (not just `app.js`): right-click the folder → *Compress* (Mac) or *Send to* → *Compressed (zipped) folder* (Windows).
3. **Test the zip.** Unzip it somewhere else, open the unzipped `index.html` with Live Server, and make sure the page still works. A missing file in the zip is the most common way to get zero.
4. Upload the .zip to Blackboard and check that the upload finished.

8. Marking scheme, and a checklist to mark yourself

The marking scheme is the same as the real midterm's:

#	Requirement	Marks
I	The riddles are read from `riddles.js` and used in the app (copying the data into your own code instead: 3 of 6)	6
II	The app shows all its components, `Header`, the nine `Riddle` cards and `Footer`, with React	14
III	Today's real date in the footer	2
IV	Proper, clean code: a loop for the riddles, short comments, no errors and no warnings	3
	Total	**25**

Marks can be taken off when a part is only partly right (for example, the design does not match, or the header or footer is missing). A page that does not show up gets 0.

Tick each box only when it is true **in the browser**, not just in your code:

☐ The page opens with Live Server and shows the blue header with the title.

☐ `index.html` loads React, ReactDOM and Babel from a CDN.

- ☐ `riddles.js` is loaded with a `<script>` tag, and its data is **not** copied into my code.
- ☐ My app has four components: `App`, `Header`, `Riddle` and `Footer`.
- ☐ Nine cards appear, numbered 1 to 9, each with its own question and answer.
- ☐ The cards come from a `map()`, not nine hand-written `<Riddle />` tags.
- ☐ Each card has a `key`, and the Console has no key warning.
- ☐ The footer shows today's real date.
- ☐ The Console shows no red errors (only the expected Babel warning).
- ☐ Each component has a short comment; there is no leftover test code.
- ☐ I unzipped my .zip in a new place and it still works.

> **WANT MORE PRACTICE?**
> Try these on your own (they are not part of the marks): show the riddles in a **random order** each time the page is refreshed; add a second line under the header that says how many riddles there are, worked out from the data; or show only the riddles whose answer starts with "A". All three need only JavaScript you already know, before the `return`.

---

9. Where to look when you are stuck

If you are stuck on…	Look at…
The CDN scripts, `type="text/babel"`, the `root` div	*Movie Night* (Module 4), its `index.html` · Code Lab Activity 1
Writing a component and using `{ }` in JSX	*Movie Night* (Module 4) · Code Lab Activity 2
Passing data with props	*Movie Night* (Module 4), the `Movie` component · Code Lab Activity 3
`map()` and `key`	*Movie Night* (Module 4), the movie list · Code Lab Activity 4
The `Date` object	MDN: search "JavaScript Date toDateString"

The React Code Lab is at **rezaghaeli.github.io/csis-3380-react-code-lab**. Activities 1 to 4 practise exactly the skills in this exam, and each one shows you the proper code after you submit.